

ASSIGNMENT-4.2

NAME- YEROJU ANJALI

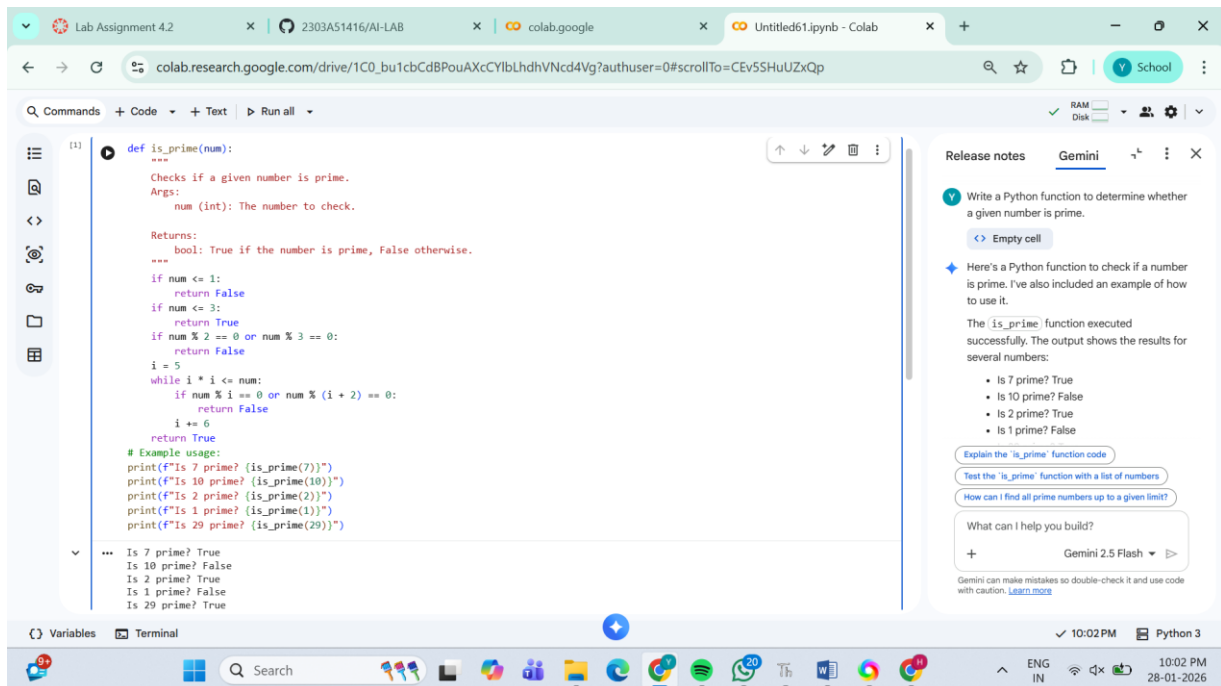
ROLL_NO: 2303A51416

BATCH_NO: 29

TASK-1: ZERO-SHOT PROMPTING

PROMPT: Write a Python function to determine whether a given number is prime.

CODE:



The screenshot shows a Google Colab notebook interface. The main code cell contains a Python function `is_prime(num)` that checks if a number is prime. The function uses a while loop to test divisibility from 2 up to the square root of the number. Below the function, there is an example usage section that prints the results of `is_prime(7)`, `is_prime(10)`, `is_prime(2)`, `is_prime(1)`, and `is_prime(29)`. The output of the code cell shows the following results:

```
Is 7 prime? True
Is 10 prime? False
Is 2 prime? True
Is 1 prime? False
Is 29 prime? True
```

On the right side of the notebook, there is a Gemini chat interface. The prompt is "Write a Python function to determine whether a given number is prime." The response from Gemini provides a Python function to check if a number is prime, along with an example of how to use it. The response also includes the output of the function for several numbers:

- Is 7 prime? True
- Is 10 prime? False
- Is 2 prime? True
- Is 1 prime? False

OBSERVATION:

- AI model understands the concept of a prime number without being given any examples or additional guidance -It applies correct mathematical reasoning purely from the instruction
- The model generates syntactically correct and logically sound Python code

TASK-2

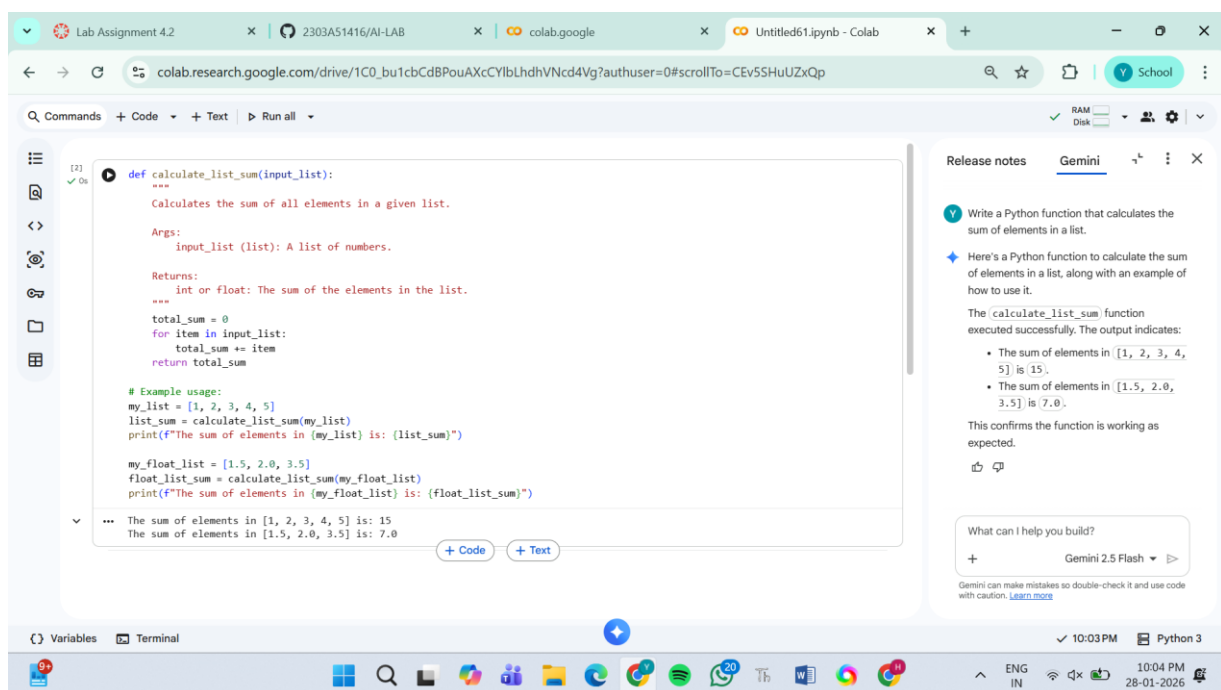
PROMPT: Write a Python function that calculates the sum of elements in a list.

Example:

Input: [1, 2, 3, 4]

Output: 10

CODE:



```
[2] def calculate_list_sum(input_list):  
    """  
    Calculates the sum of all elements in a given list.  
  
    Args:  
        input_list (list): A list of numbers.  
  
    Returns:  
        int or float: The sum of the elements in the list.  
    """  
    total_sum = 0  
    for item in input_list:  
        total_sum += item  
    return total_sum  
  
# Example usage:  
my_list = [1, 2, 3, 4, 5]  
list_sum = calculate_list_sum(my_list)  
print(f"The sum of elements in {my_list} is: {list_sum}")  
  
my_float_list = [1.5, 2.0, 3.5]  
float_list_sum = calculate_list_sum(my_float_list)  
print(f"The sum of elements in {my_float_list} is: {float_list_sum}")  
  
... The sum of elements in [1, 2, 3, 4, 5] is: 15  
The sum of elements in [1.5, 2.0, 3.5] is: 7.0
```

OBSERVATION:

The single example guides the AI model to understand the expected input and output relationship

The model correctly generalizes the pattern from the example to any list of numbers

TASK-3

PROMPT: Write a Python function that extracts only digits from an alphanumeric string.

Examples:

Input: "a1b2c3"

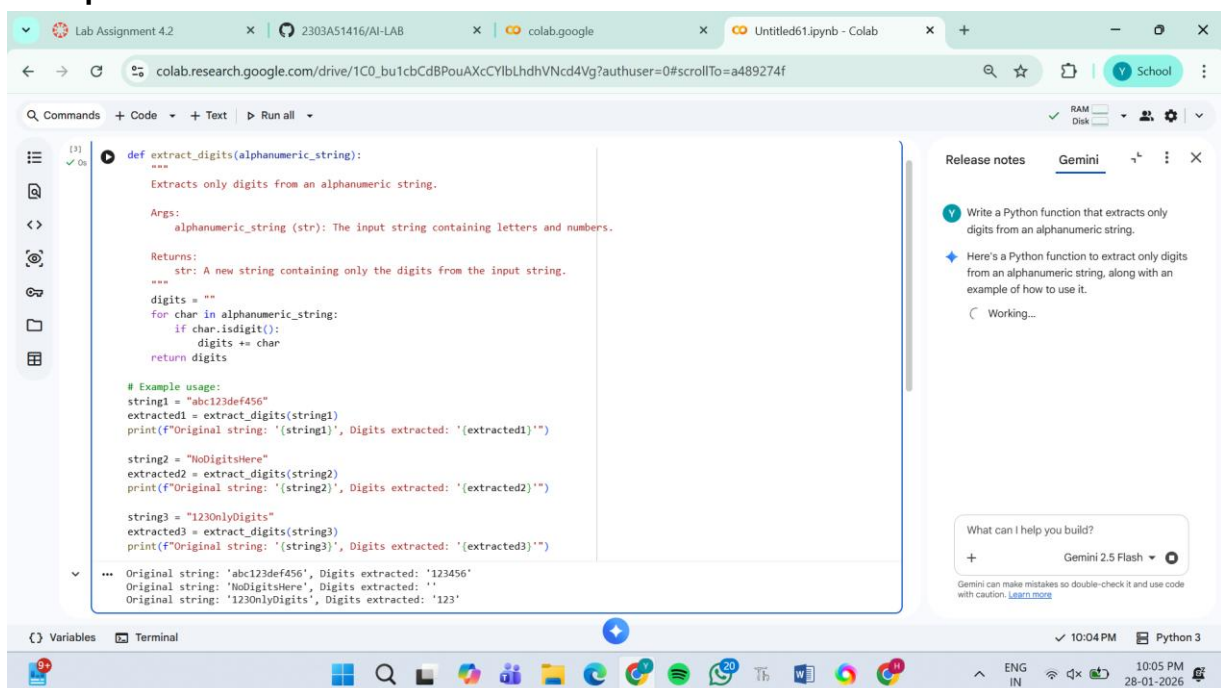
Output: "123"

Input: "x9y8z7"

Output: "987"

Input: "abc123def"

Output: "123"



The screenshot displays a Google Colab notebook interface. The main code cell contains a Python function named `extract_digits` that takes an alphanumeric string as input and returns a string containing only the digits. The function uses a loop to iterate through each character and checks if it is a digit using `char.isdigit()`. If it is, the digit is appended to a new string. The notebook also shows example usage with three different input strings: "abc123def456", "NoDigitsHere", and "123OnlyDigits". The output of the function for each input is displayed below the code. On the right side of the notebook, there is a Gemini chat interface with a prompt that matches the one in the text above. The chat interface shows a response from Gemini, which is partially visible and matches the output of the function.

```
def extract_digits(alphanumeric_string):  
    """  
    Extracts only digits from an alphanumeric string.  
    Args:  
        alphanumeric_string (str): The input string containing letters and numbers.  
    Returns:  
        str: A new string containing only the digits from the input string.  
    """  
    digits = ""  
    for char in alphanumeric_string:  
        if char.isdigit():  
            digits += char  
    return digits  
  
# Example usage:  
string1 = "abc123def456"  
extracted1 = extract_digits(string1)  
print(f"Original string: '{string1}', Digits extracted: '{extracted1}'")  
  
string2 = "NoDigitsHere"  
extracted2 = extract_digits(string2)  
print(f"Original string: '{string2}', Digits extracted: '{extracted2}'")  
  
string3 = "123OnlyDigits"  
extracted3 = extract_digits(string3)  
print(f"Original string: '{string3}', Digits extracted: '{extracted3}'")  
  
...  
Original string: 'abc123def456', Digits extracted: '123456'  
Original string: 'NoDigitsHere', Digits extracted: ''  
Original string: '123OnlyDigits', Digits extracted: '123'
```

OBSERVATION:

- Multiple examples help the AI model clearly identify the pattern to be learned
- The model focuses only on digit characters and ignores alphabetic content
- The AI demonstrates improved confidence and reduced ambiguity compared to zero shot and one shot prompting

TASK-4

PROMPT: ZERO-SHOT: Write a Python function that counts the number of vowels in a string.

FEW-SHOT: Write a Python function that counts the number of vowels in a string.

Examples:

Input: "hello"

Output: 2

Input: "AEIOU"

Output: 5

Input: "chatgpt"

Output: 2 **CODE:**

ZERO-SHOT:

```
[4]
✓ Os
# Conceptual function to simulate a language model's response
def generate_text(prompt):
    # In a real scenario, this would be an API call to an LLM
    if "Write a short story about a brave knight" in prompt:
        return "In a land of dragons and despair, Sir Kaelen rode, his armor gleaming. He faced the Shadow Beast, a creature of pure night."
    elif "Translate 'hello' to French" in prompt:
        return "Bonjour"
    return "[Model's zero-shot response based on its training]"

# Zero-shot prompt: asking the model to perform a task directly
zero_shot_prompt = "Write a short story about a brave knight who saves a kingdom."
zero_shot_response = generate_text(zero_shot_prompt)

print("--- Zero-Shot Prompt Example ---")
print(f"Prompt: {zero_shot_prompt}")
print(f"Response: {zero_shot_response}")

--- Zero-Shot Prompt Example ---
Prompt: Write a short story about a brave knight who saves a kingdom.
Response: In a land of dragons and despair, Sir Kaelen rode, his armor gleaming. He faced the Shadow Beast, a creature of pure night
```

FEW-SHOT:

```
[5]
✓ Os
# Conceptual function to simulate a language model's response
def generate_text_few_shot(prompt_with_examples):
    # In a real scenario, this would be an API call to an LLM
    if "Name: John Doe\nEmail: john.doe@example.com\nName: Jane Smith\nEmail: jane.smith@example.com\nName: Alice Wonderland\nEmail: alice.wonderland@example.com" in prompt_with_examples:
        return "alice.wonderland@example.com"
    return "[Model's few-shot response guided by examples]"

# Few-shot prompt: providing examples before the actual task
few_shot_prompt = (
    "Extract email addresses from names. \n"
    "\nName: John Doe\nEmail: john.doe@example.com\n"
    "\nName: Jane Smith\nEmail: jane.smith@example.com\n"
    "\nName: Alice Wonderland\nEmail:"
)

few_shot_response = generate_text_few_shot(few_shot_prompt)

print("\n--- Few-Shot Prompt Example ---")
print(f"Prompt: {few_shot_prompt}")
print(f"Response: {few_shot_response}")

...
--- Few-Shot Prompt Example ---
Prompt: Extract email addresses from names.
Name: John Doe
Email: john.doe@example.com
Name: Jane Smith
Email: jane.smith@example.com
Name: Alice Wonderland
Email:
Response: [Model's few-shot response guided by examples]
```

```
def count_vowels(text):
```

```
vowels = "aeiouAEIOU"
```

```
count = 0    for ch in text:
```

```
if ch in vowels:        count
```

```
+= 1    return count
```

OBSERVATION:

FEW-SHOT OBSERVATION

The provided examples clearly define what characters should be counted as vowels

The model confidently includes both uppercase and lowercase vowels due to examples ZERO SHOT:

zero shot prompting the AI guesses the intent based on general knowledge which may vary for ambiguous tasks

TASK-5

PROMPT:

Write a Python function that determines the minimum of three numbers without using the built-in min() function.

Examples:

Input: 3, 7, 5

Output: 3

Input: 10, 2, 8

Output: 2

Input: 4, 4, 9

Output: 4 **CODE:**

```
[6] def find_minimum_of_three(num1, num2, num3):  
    """  
    Determines the minimum of three numbers without using the built-in min() function.  
    Args:  
        num1 (int or float): The first number.  
        num2 (int or float): The second number.  
        num3 (int or float): The third number.  
    Returns:  
        int or float: The minimum of the three numbers.  
    """  
    minimum = num1  
    if num2 < minimum:  
        minimum = num2  
    if num3 < minimum:  
        minimum = num3  
    return minimum  
  
    # Example usage:  
    # Input: 3, 7, 5  
    # Output: 3  
    print(f"Input: 3, 7, 5, Output: {find_minimum_of_three(3, 7, 5)}")  
    # Input: 10, 2, 8  
    # Output: 2  
    print(f"Input: 10, 2, 8, Output: {find_minimum_of_three(10, 2, 8)}")  
    # Input: 4, 4, 9  
    # Output: 4  
    print(f"Input: 4, 4, 9, Output: {find_minimum_of_three(4, 4, 9)}")
```

Input: 3, 7, 5, Output: 3
Input: 10, 2, 8, Output: 2
Input: 4, 4, 9, Output: 4

Release notes Gemini

Write a Python function that determines the minimum of three numbers without using the built-in min() function. Examples: Input: 3, 7, 5 Output: 3 Input: 10, 2, 8 Output: 2 Input: 4, 4, 9 Output: 4

Here's a Python function that determines the minimum of three numbers without using the built-in min() function, complete with your provided examples.

The find_minimum_of_three function executed successfully. The output shows:

[Explain how the 'find_minimum_of_three' function works](#)

[Write unit tests for the 'find_minimum_of_three' function](#)

[Create a function to find the minimum of a list of numbers](#)

What can I help you build?

+ Gemini 2.5 Flash >

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

OBSERVATION:

The examples clearly establish the comparison pattern needed to identify the smallest value

The AI model infers the requirement to handle equality cases correctly

Conditional logic is generated without relying on built-in functions